Institut Supérieur d'Informatique
Département génie télécommunication et réseaux (GTR)

Module : Cloud Computing & Virtualisation

Groupes : M1 SSII

Enseignant : Safa Réjichi

Nombre de pages : 5

@mail : safa.rejichi@isi.utm.tn

Travaux Pratiques 2 : Conteneurisation d'une application Web

Objectifs :

- ⇨ Création d'une stack LAMP au moyen de Docker.
- ⇨ Manipulation de Dockerfile
- ⇨ Manipulation de volumes
- ⇨ Appréhension de la notion de réseau avec docker

Partie 1 : Manipulation de Dockerfile

Pour s'exercer, nous allons créer notre propre stack LAMP (Linux Apache MySQL PHP) au moyen de Docker. Voici les différentes couches de cette image :

- Une couche OS pour exécuter notre Apache, MySQL et Php, nous allons nous baser sur la distribution Ubuntu.
- Une couche Apache pour démarrer notre serveur web.
- Une couche php qui contiendra un interpréteur Php mais aussi les bibliothèques qui vont avec.
- Une couche Mysql qui contiendra notre système de gestion de bases de données.

Voici le schéma de notre image :

I. Normalement pour faire les choses dans les règles de l'art, il faut séparer l'image de l'application web par rapport à l'image de la base de données. Mais pour ce TP, nous allons faire une exception et nous allons mettre toute notre stack dans une seule image.

1. Ouvrez la machine virtuelle fournie avec ce fascicule. Choisissez un répertoire sur cette MV, par exemple « Documents », et créez un dossier portant votre nom_prénom.
2. Désarchivez le fichier zip fourni avec ce fascicule de TP. Vous trouverez les dossiers suivants :
   - db : contient un fichier articles.sql, qui renferme toute l'architecture de la base de données.
   - app : comporte les sources php de notre l'application web.
3. Ensuite dans votre répertoire, créez un fichier et nommez le Dockerfile. Rajoutez le contenu suivant, afin de créer la stack LAMP (n'oubliez pas de remplacer `NOM Prénom <@votre_adresse_mail` par vos données personnelles) :

```
# --------------- DÉBUT COUCHE OS -------------------
FROM ubuntu:latest
# --------------- FIN COUCHE OS ---------------------

# MÉTADONNÉES DE L'IMAGE
LABEL version="1.0" maintainer="NOM Prénom <@votre_adresse_mail>"


# VARIABLES TEMPORAIRES
ARG APT_FLAGS="-q -y"
ARG DOCUMENTROOT="/var/www/html"

# --------------- DÉBUT COUCHE APACHE ---------------
RUN apt-get update -y && DEBIAN_FRONTEND=noninteractive apt-get install ${APT_FLAGS} apache2
# --------------- FIN COUCHE APACHE -----------------

# --------------- DÉBUT COUCHE MYSQL ----------------
RUN apt-get install ${APT_FLAGS} mariadb-server

COPY sources/db/articles.sql /
# --------------- FIN COUCHE MYSQL ------------------

# --------------- DÉBUT COUCHE PHP ------------------
RUN DEBIAN_FRONTEND=noninteractive apt-get install ${APT_FLAGS} \
    php-mysql \
    php && \
    rm -f ${DOCUMENTROOT}/index.html && \
    apt-get autoclean -y

COPY sources/app ${DOCUMENTROOT}
# --------------- FIN COUCHE PHP --------------------

# OUVERTURE DU PORT HTTP
EXPOSE 80

# RÉPERTOIRE DE TRAVAIL
WORKDIR  ${DOCUMENTROOT}

# DÉMARRAGE DES SERVICES LORS DE L'EXÉCUTION DE L'IMAGE
ENTRYPOINT service mariadb start && mysql < /articles.sql && apache2ctl -D FOREGROUND
```

Voici l'architecture que vous êtes censé avoir :

```
Documents
├──sources
│     ├──app
│     │   ├── db-config.php
│     │   ├── index.php
│     │   └── validation.php
│     └── db
│         └── articles.sql
└── Dockerfile
```

4. Expliquez le contenu du Dockerfile ?

II. Placez-vous dans le répertoire « Documents/nom_prénom » et construisez votre image avec le Dockerfile.

1. Construisez votre image que vous nommerez « my_lamp » avec la commande docker build -t <IMAGE_NAME> . :

docker build -t my_lamp .

2. Vérifiez que votre image a été créée par la commande :

docker images

III. Création du conteneur :

1. Exécutez votre conteneur que vous nommerez « my_lamp_c » avec la commande :

docker run -d --name my_lamp_c -p 8080:80 my_lamp

2. Expliquez l'option -p 8080 :80 ?
3. Afin de donner les droits nécessaires aux fichiers liés à notre serveur web, exécutez les commandes suivantes :

docker exec my_lamp_c chmod 777 index.php
docker exec my_lamp_c chmod 777 db-config.php
docker exec my_lamp_c chmod 777 validation.php

4. Vérifiez que votre conteneur est en cours d'exécution par la commande :

docker ps

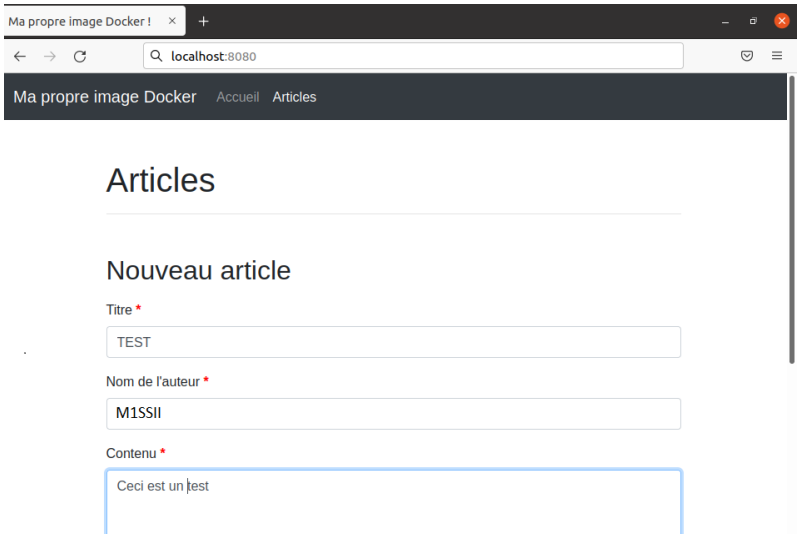
5. S'il n'est pas en cours d'exécution vous pouvez afficher les erreurs. Dès fois, vous aurez besoin de déboguer votre conteneur en regardant les sorties/erreurs du conteneur par la commande :

docker logs -ft my_lamp_c

6. Visitez ensuite la page suivante http://localhost:8080/, et affichez le résultat obtenu.

IV. Utilisation de la stack lamp.

1. Ajoutez une ressource (Nouveau article) avec les informations suivantes :



2. Détruisez votre conteneur « my_lamp_c » et l'exécutez une autre fois, avec les commandes :

```
docker stop my_lamp_c
docker rm my_lamp_c
docker run -d --name my_lamp_c -p 8080:80 my_lamp
docker exec my_lamp_c chmod 777 -R /var/www/html
```

3. Vérifiez l'existence de l'article déjà ajouté. Expliquez ?

Partie 2 : Manipulation de Volume

Afin de pouvoir sauvegarder (persister) les données et également partager des données entre conteneurs, Docker a développé le concept de volumes. Les volumes sont des répertoires (ou des fichiers) qui ne font pas partie du système de fichiers UFS (Union File System) mais qui existent sur le système de fichiers hôte.

I. Pour créer un volume, afin de stocker notre base de données :
1. Créez un volume nommé `mysqldata` avec la commande :
   `docker volume create --name mysqldata`
2. Pour lister les volumes créer tapez:
   `docker volume ls`
3. Exécutez votre conteneur que vous nommerez « my_lamp_c » avec la commande :
   `docker run -d --name my_lamp_c -v mysqldata:/var/lib/mysql -p 8080:80 my_lamp`
4. Expliquez l'option `-v mysqldata:/var/lib/mysql` ?

II. Visitez ensuite la page suivante http://localhost:8080/, et ajoutez le même article avec les mêmes informations.

III. Détruisez votre conteneur « my_lamp_c » et l'exécutez une autre fois.

IV. Vérifiez l'existence de l'article ajouté. Expliquez ?

Partie 3 : Manipulation du Réseau

Pour que les conteneurs Docker puissent communiquer entre eux mais aussi avec le monde extérieur via la machine hôte, alors une couche de mise en réseau est nécessaire. Cette couche réseau rajoute une partie d'isolation des conteneurs, et permet donc de créer des applications Docker qui fonctionnent ensemble de manière sécurisée.

I. Vérifiez qu'un réseau bridge est créé par défaut, par la commande :
   `docker network ls`

II. Plus précisément c'est l'interface docker0 au niveau de la machine hôte.
1. Donnez l'adresse ip de la machine hôte.
2. Affichez l'adresse ip de votre conteneur :
   `docker exec my_lamp_c ip add`
3. Vérifiez qu'ils sont bien sur le même réseau.

Partie 4 : Publier son image dans le Hub Docker

Si vous souhaitez partager votre image avec d'autres utilisateurs, une des possibilités est d'utiliser le Hub Docker.

I. Vous allez partager l'image « my_lamp ».
   1. Pour cela, commencez par visiter le site web https://hub.docker.com/. Puis, inscrivez-vous sur la plateforme et créez ensuite un repository public. Une fois que vous aurez choisi le nom et la description de votre repository, cliquez ensuite sur le bouton create.
   2. L'étape suivante est de se connecter au hub Docker à partir de la ligne de commande :
      docker login
   3. Il va vous demander, votre nom d'utilisateur et votre mot de passe, et si tout se passe bien vous devez avoir le message suivant :
      Login Succeeded

II. Préparez votre image.
   1. Récupérer l'id ou le nom de votre image :
      docker images
   2. Ensuite il faut rajouter un tag à l'id ou le nom de l'image récupérée par la commande :
      docker tag <IMAGENAME OU ID> <HUB-USER>/<REPONAME>[:<TAG>]
      Vous pouvez utiliser comme tag :first ou même un numéro :0.1.
   3. Relancez la commande docker images, afin de voir votre image avec le bon tag.

III. Maintenant envoyez votre image, en la pushant vers le Hub Docker grâce à la commande suivante :
   docker push <HUB-USER>/<REPONAME>[:<TAG>]

IV. Vérifiez que l'image a été bien stockée dans le Hub Docker.